LECTURE 09

Object Arrays, Composition & Class Design

Object arrays • reference sharing • HAS-A relationships • passing/returning objects

From isolated classes → collaborating object structures



CSC241 — Object-Oriented Programming • Fall 2026 • COMSATS University Islamabad, Lahore Campus
Instructor: Muhammad Shahid Bhatti

Bridge from Lecture 8

CONTINUITY

Lecture 8 — Discover

Identify candidate classes from requirements.

Assign responsibilities and collaborators.

Sketch an initial conceptual class diagram.

Lecture 9 — Connect + Organize

Organize multiple objects using object arrays, then connect objects using references.

Pass/return objects and arrays while tracing reference behavior.

Build HAS-A relationships and distinguish them from IS-A inheritance.

Lecture 10 — Deepen

Multi-level composition.

Cohesion, coupling, ownership, and lifetime dependency.

Association, aggregation, composition, and cloning.

Today: move from single-object references to arrays of object references, then to collaborating object structures.

Lecture 9 scope from the course plan

CDF ALIGNMENT

1 HAS-A relationships

One object keeps a reference to another object.

The containing object can delegate behavior to its component.

2 Passing & returning objects

Objects can be method arguments.

Methods can return references to objects.

3 Composition vs inheritance

HAS-A models collaboration/containment.

IS-A models specialization.

Choose based on domain meaning, not code reuse alone.

0 Object Arrays

Arrays store references

Fixed-size collections

No ArrayList yet

Formal association / aggregation / composition notation is intentionally deferred to Lecture 10–11.

Learning outcomes

OUTCOMES

Recognize

Explain a HAS-A relationship between two domain objects.

Implement

Represent object relationships using reference fields and constructors.

Pass

Pass an object reference to a method and predict what the method can change.

Return

Return object references while preserving encapsulation decisions.

Choose

Defend composition vs. inheritance for a given design.

Compile

Create and run a small collaborating-object program in live coding.

ARRAYS
Create + trace object arrays

The big idea: classes rarely work alone

MOTIVATION

Real OO systems are graphs of collaborating objects, not isolated class definitions.



Each relationship should answer a domain question: “Why does this object need to know that object?”

What does “composition” mean in everyday OO discussion?

HAS-A

Core idea

A class contains a field whose type is another class.

The containing object can use that referenced object to perform work.

The relationship is read naturally as “A has a B”.

Example: CourseSection has a Course object.

JAVA

```
class CourseSection {
    private Course course;
    private int capacity;

    CourseSection(Course course, int capacity) {
        this.course = course;
        this.capacity = capacity;
    }
}
```

Today we use “composition” broadly as object composition / HAS-A. Strong lifetime semantics come in Lecture 10.

Why use object composition?

DESIGN BENEFIT

Model the domain

A CourseSection should refer to the Course it offers.

A Registration may connect a Student and CourseSection.

Reuse behavior

The containing object can delegate to the component instead of copying its data or logic.

One Course object can serve multiple sections.

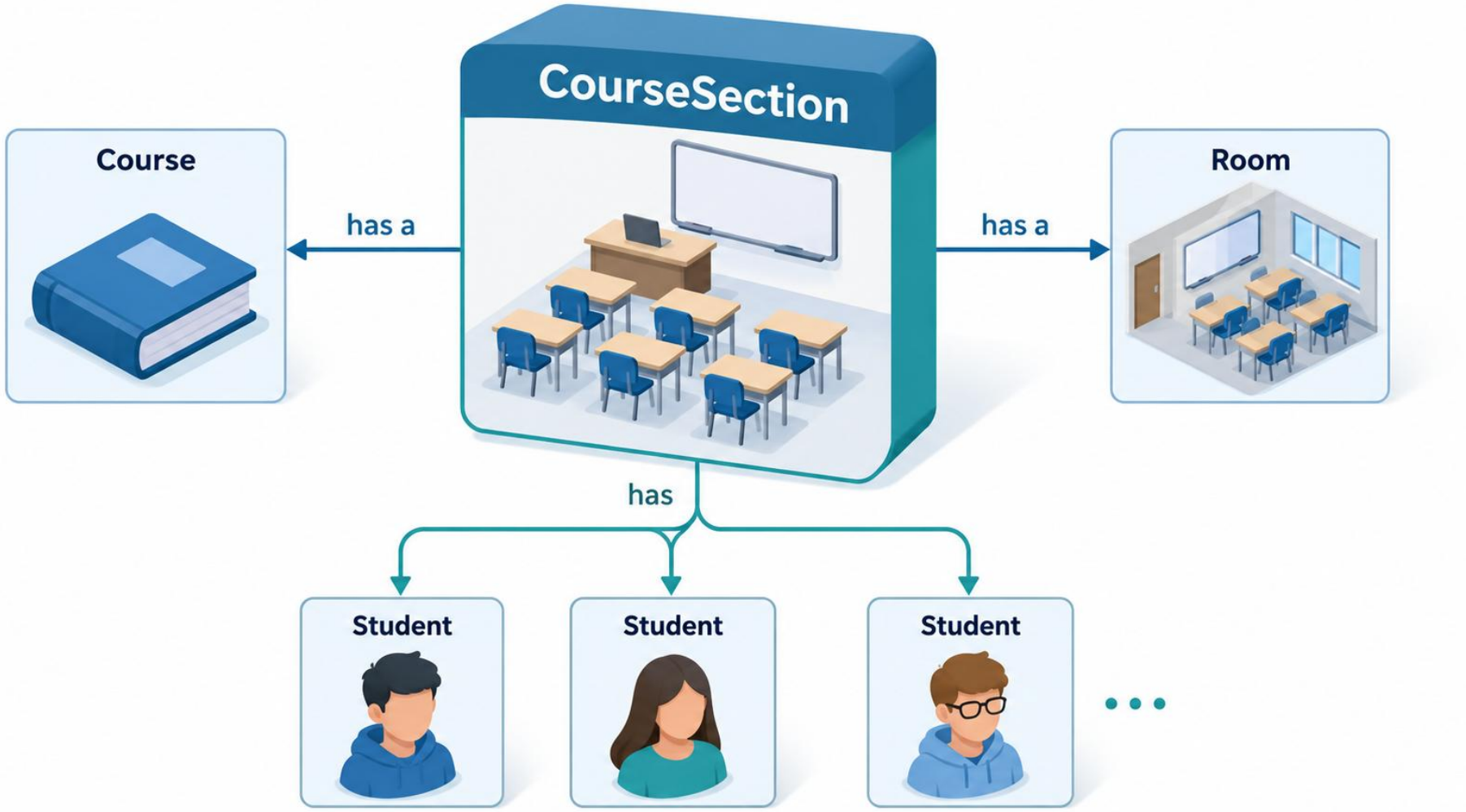
Localize change

If Course changes how it formats its title, CourseSection can keep using course.getDisplayName().

Fewer duplicated fields means fewer consistency bugs.

Running example — CourseSection has related objects

VISUAL MODEL



Reference fields connect objects

OBJECT REFERENCES

COURSE.JAVA

```
class Course {
    private String code;
    private String title;

    Course(String code, String title) {
        this.code = code;
        this.title = title;
    }

    String getCode() { return code; }
}
```

COURSESECTION.JAVA

```
class CourseSection {
    private Course course;   // object reference
    private String section;

    CourseSection(Course course, String section) {
        this.course = course;
        this.section = section;
    }
}
```

Live coding checkpoint 1 — build the two classes

LIVE CODING

Predict → Code

Create Course.java with code and title.

Create CourseSection.java with a Course field.

Ask students: does CourseSection contain a copy of Course or a reference to the same object?

Compile before adding extra methods.

TERMINAL

```
javac Course.java CourseSection.java

// No main method yet.
// Successful compilation means the type relationship is valid.
```

Teaching move: intentionally misspell Course once and use the compiler error to discuss type references.

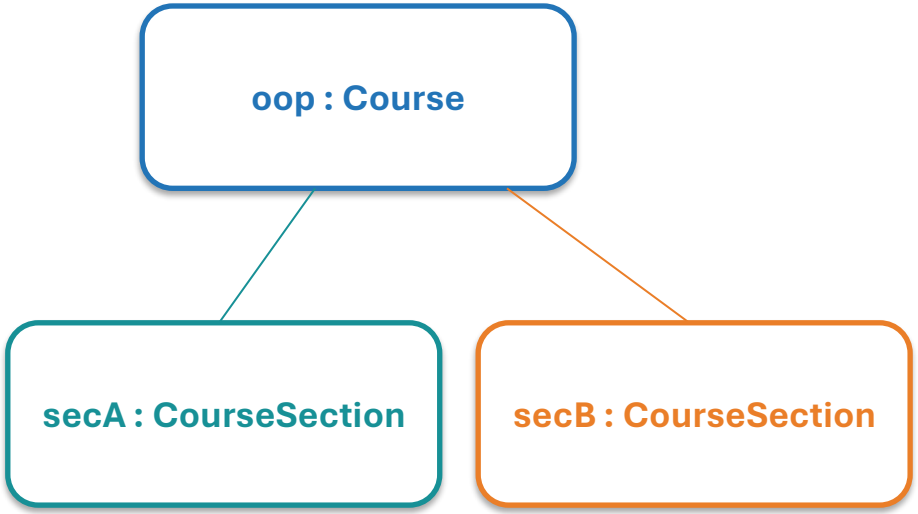
Create objects, then wire them together

MAIN.JAVA

```
Course oop = new Course(
    "CSC241",
    "Object-Oriented Programming"
);

CourseSection secA = new CourseSection(oop, "A");
CourseSection secB = new CourseSection(oop, "B");
```

OBJECT GRAPH



Two sections can reference the same Course object.

Delegate instead of duplicating data

DELEGATION

Weak design — duplicate course data

CourseSection stores courseCode and courseTitle itself.

If title changes, every section copy must be updated.

State can become inconsistent.

BETTER

```
String getLabel() {
    return course.getCode() + "-" + section;
}

// CourseSection asks Course for course
data.
// It does not duplicate Course's internal
state.
```

Composition often reduces duplicate state by letting objects ask collaborators for what they need.

Constructor injection makes required relationships explicit

JAVA

```
class CourseSection {
    private final Course course;
    private final String section;

    CourseSection(Course course, String section) {
        this.course = course;
        this.section = section;
    }
}
```

Why this is useful

A CourseSection cannot be created without a Course reference.

The dependency is visible in the constructor signature.

final prevents the field from being reassigned later.

The constructor establishes the relationship at object creation.

Passing objects to methods

COURSESECTION.JAVA

```
class CourseSection {
    private int enrolled;
    private int capacity;

    boolean enroll(Student student) {
        if (student == null || enrolled >= capacity)
            return false;

        enrolled++;
        System.out.println(student.getName() + " enrolled");
        return true;
    }
}
```

What is passed?

The method receives a copy of the reference value.

Both caller and parameter can refer to the same Student object.

The method may call Student methods through that reference.

Java still uses pass-by-value.

Live example — pass a real Student object

LIVE CODE

MAIN.JAVA

```
Student a = new Student(
    "CIIT/SP26-BAI-005/LHR",
    "Abeer Amina"
);

CourseSection secA = new CourseSection(oop, "A", 40);
boolean ok = secA.enroll(a);
System.out.println(ok);
```

Ask before running

Which object does parameter student refer to?

Does enroll() create another Student object?

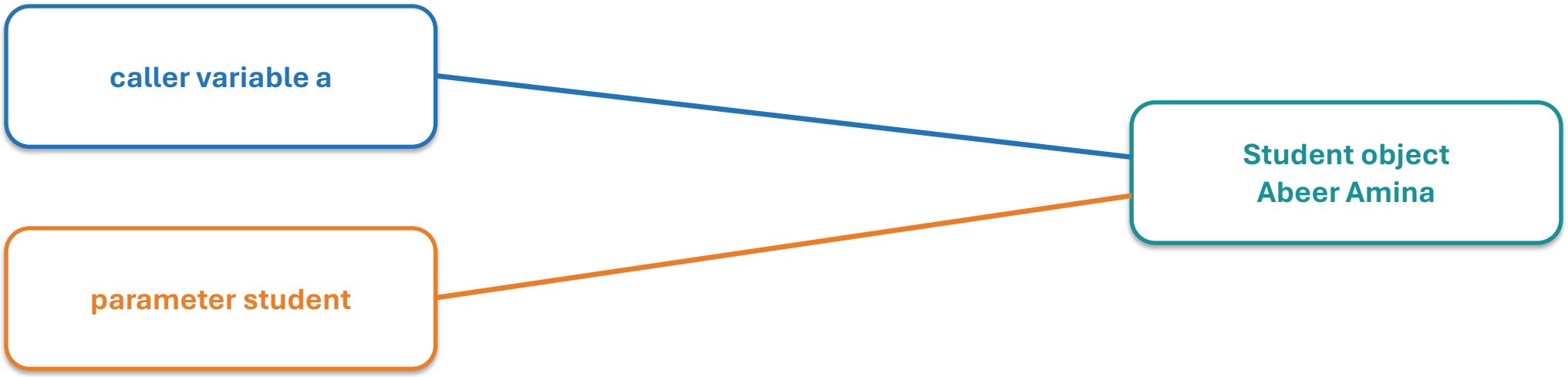
What changes if section capacity is already full?

What happens if a is null?

Java passes object references by value

IMPORTANT

The reference is copied; the object is not copied.



Two reference values → one object

Mutation vs. reassignment inside a method

PASS-BY-VALUE

JAVA

```
static void rename(Student s) {
    s.setName("Updated Name");
}

static void replace(Student s) {
    s = new Student("X", "Another Student");
}
```

rename(s)

Mutates the shared object.

Caller observes the changed name.

replace(s)

Only reassigns the local parameter variable.

Caller still holds its original reference.

Returning objects from methods

JAVA

```
class CourseSection {
    private Course course;

    Course getCourse() {
        return course;
    }
}

Course returned = secA.getCourse();
System.out.println(returned.getCode());
```

What is returned?

A reference value to an existing Course object.

No automatic object copy occurs.

The caller may now collaborate with that Course object.

Returning references is powerful—but affects encapsulation choices.

Return a collaborator — but decide what clients may do

ENCAPSULATION

Returning the object can be appropriate

Course is intended to be a public domain collaborator.

Clients need meaningful Course behavior.

Course protects its own invariants.

Returning internal mutable state can be risky

A client may obtain a reference and modify state you wanted controlled.

Do not automatically create getters for every object field.

Expose behavior or read-only information according to the class API.

Encapsulation still matters after objects are composed.

Required collaborator or null?

VALID STATE

JAVA

```
CourseSection(Course course, String section) {
    if (course == null)
        throw new IllegalArgumentException("course
required");

    this.course = course;
    this.section = section;
}
```

Design question

Can a valid CourseSection exist without a Course?

If no, reject null at construction.

A relationship is part of the object invariant when the component is required.

Passing a composed object through methods

OBJECT FLOW

JAVA

```
static void printSection(CourseSection s) {
    Course c = s.getCourse();
    System.out.println(c.getCode());
}

printSection(secA);
```

Trace the references

main holds secA.

printSection receives a copy of secA's reference.

secA holds a Course reference.

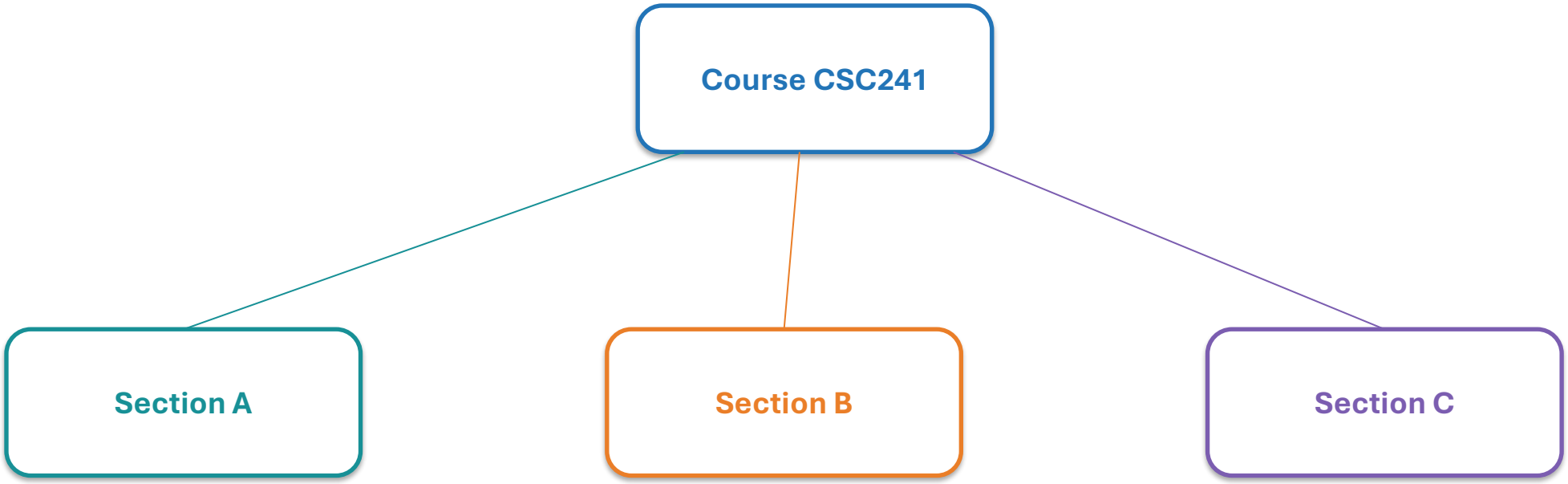
getCourse() returns a copy of that Course reference.

No domain object is copied by these assignments.

One object can be shared by several owners of references

SHARED REFERENCE

Sharing a collaborator is normal when the domain says the same object is being referenced.



Do not create three duplicate Course objects merely because there are three sections.

Why object arrays belong here

CDF alignment + a natural bridge from references to object collaboration

CDF / syllabus
Object Arrays are an explicit early OOP topic

What students already know
objects + references + constructors + loops

Next design step
many objects → one structured group

Today we use arrays as a fixed-size way to organize object references — then composition explains how objects collaborate.

Primitive array vs. object array

Same array syntax — very different element meaning

int[] marks = new int[3];

0	0	0

elements hold primitive values

Student[] students = new Student[3];

null	null	null

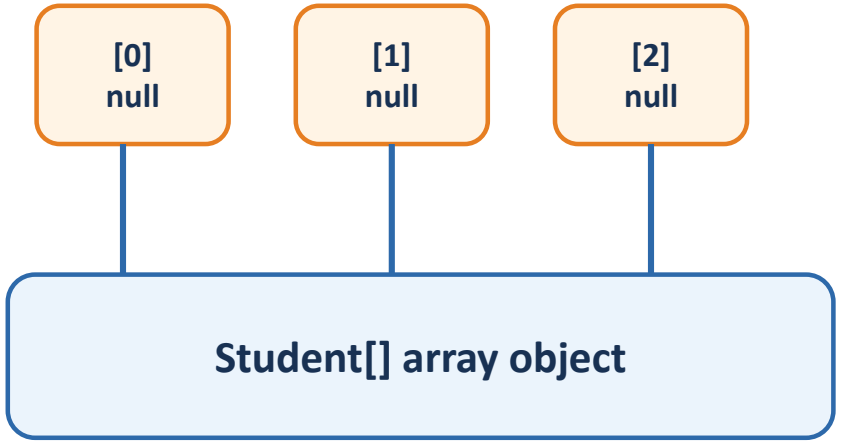
elements hold reference values

IMPORTANT

An object array does not create the objects

new Student[3] creates one array object containing three reference slots

```
Student[] students =
    new Student[3];
```



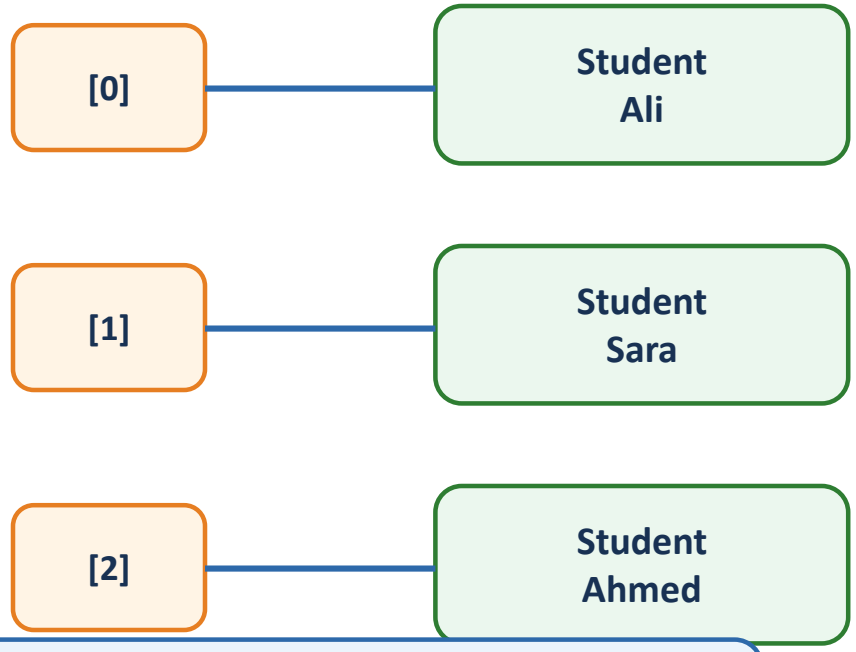
No Student object exists yet. Each element is initially null until you store a Student reference in that slot.

Fill the object array with real objects

Each assignment stores a reference in one array element

```
Student[] students = new Student[3];

students[0] = new Student("BCS-001", "Ali");
students[1] = new Student("BCS-002", "Sara");
students[2] = new Student("BCS-003", "Ahmed");
```



Array elements contain references — the Student objects remain separate heap objects.

Array initializer: shorter syntax, same reference model

Three expressions create three Student objects; the array stores three references

```
Student[] students = {
new Student("BCS-001", "Ali"),
new Student("BCS-002", "Sara"),
new Student("BCS-003", "Ahmed")
};
```

students.length == 3

students[0] → Student object

students[3] → ERROR

Valid indexes: 0 to length - 1

Iterate through an object array

A loop variable receives each stored reference one at a time

```
for (int i = 0; i < students.length; i++) {
            System.out.println(
             students[i].getName()
                    );
                  }
```

```
for (Student s : students) {
      System.out.println(
            s.getName()
              );
             }
```

index-based loop
useful when position matters

enhanced for loop
useful for simple traversal

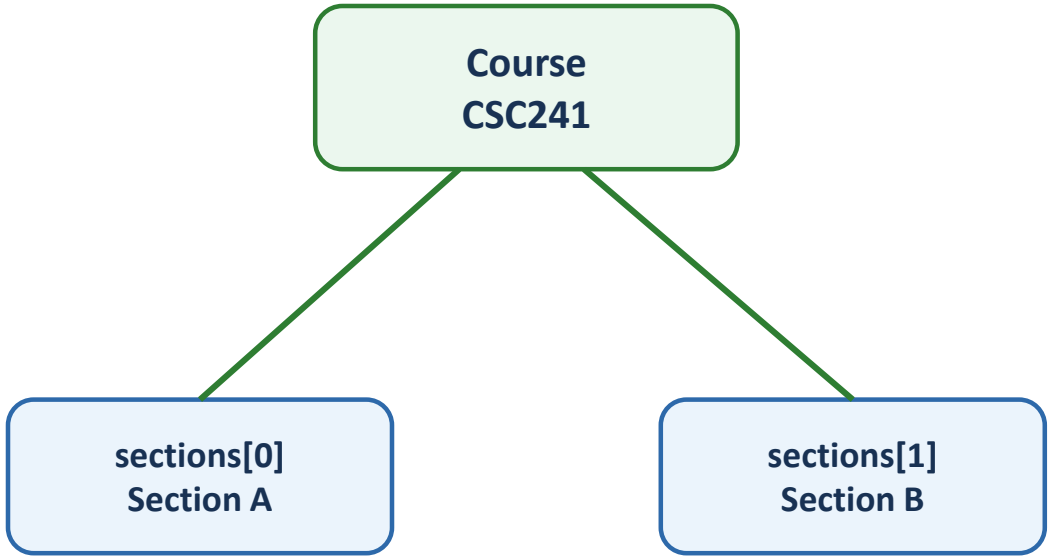
Always consider possible null elements before calling a method.

Shared references inside object arrays

Arrays do not prevent several objects from sharing one collaborator

```
Course oop = new Course("CSC241", "OOP");

     CourseSection[] sections = {
       new CourseSection(oop, "A"),
        new CourseSection(oop, "B")
                  };
```



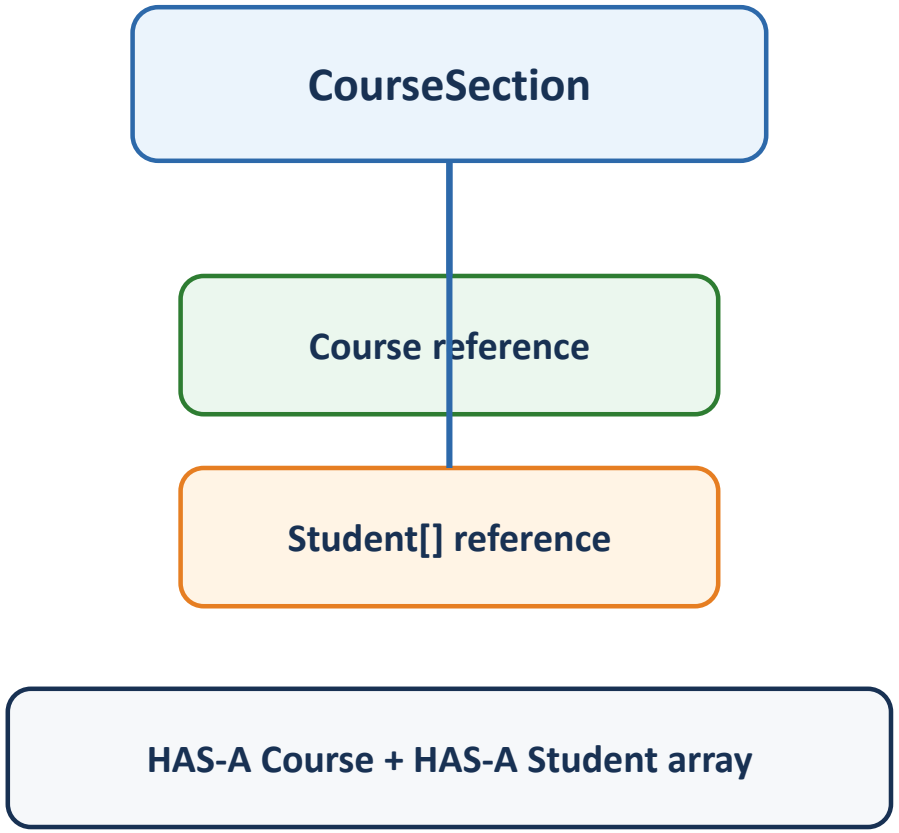
One Course object; two CourseSection objects; one array object; both sections share the same Course reference.

Composition with an array field

A class can HAS-A collection of references using a fixed-size array

```
class CourseSection {
  private Course course;
private Student[] students;
    private int count;

CourseSection(Course course, int capacity) {
            this.course = course;
       students = new Student[capacity];
                count = 0;
                  }
                 }
```



Fixed-capacity enrollment without ArrayList

Arrays let us build a simple roster using only concepts students already know

```
boolean enroll(Student student) {
if (student == null || count >= students.length)
return false;

students[count] = student;
count++;
return true;
}

void printRoster() {
for (int i = 0; i < count; i++) {
System.out.println(students[i].getName());
}
}
```

capacity = students.length

count = number actually enrolled

students[count] = next free slot

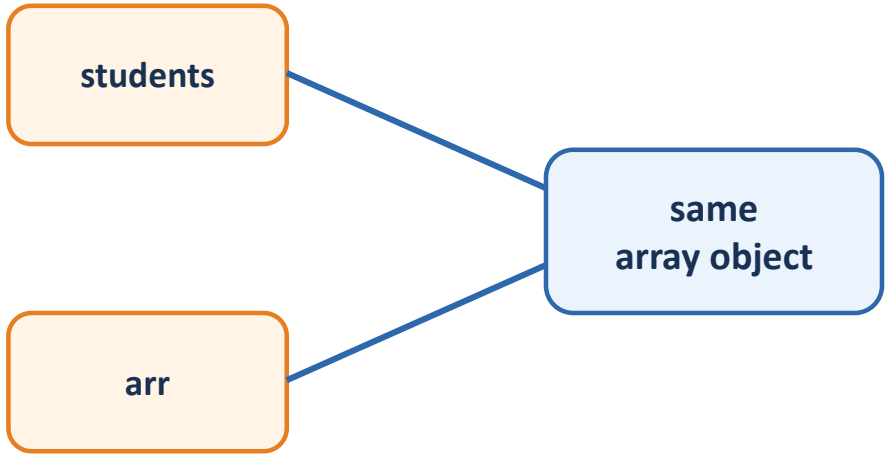
No ArrayList required

Passing an object array to a method

The parameter receives a copy of the array reference value

```
static void renameFirst(Student[] arr) {
        arr[0].setName("Updated");
                }

        renameFirst(students);
```



Both references reach the same array; arr[0] reaches the same Student object as students[0].

Java still passes the reference value by value.

Returning an array can expose internal state

Encapsulation still matters when the field is an array

```
Student[] getStudents() {
        return students;
             }

    // caller can now do:
section.getStudents()[0] = null;
```

Risk: caller can replace array elements

Safer: expose behavior

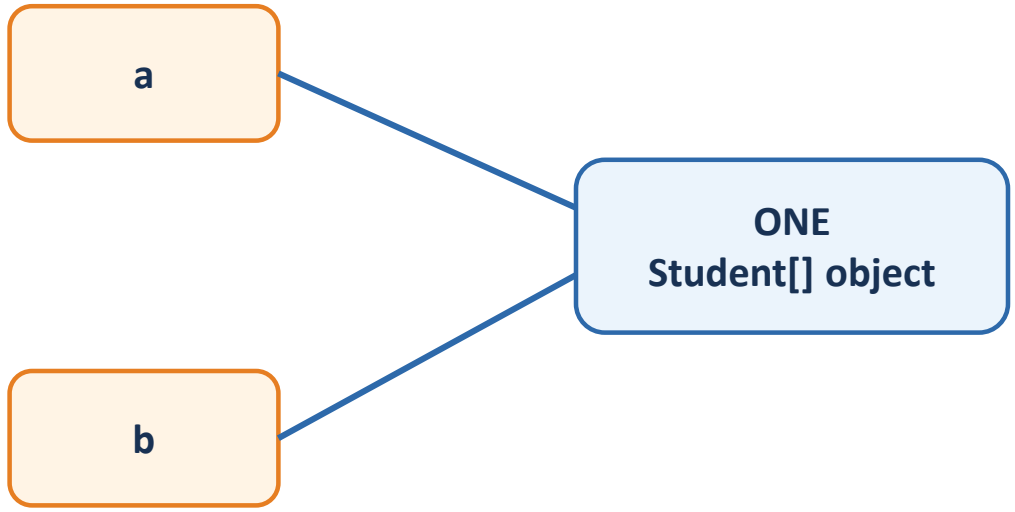
```
Student getStudent(int index)
int getEnrolledCount()
void printRoster()
```

Do not automatically return internal arrays just because a getter is easy to write.

Array assignment is NOT array copying

This prepares the mental model for shallow vs. deep copy later

```
Student[] a = students;
Student[] b = a;
```



Changing b[0] also changes what a[0] sees because a and b refer to the same array object.

Next lecture: copying a composed object or object array → shallow copy vs. deeper copy decisions.

Live coding checkpoint — section roster

Build one small program that combines arrays + object references + composition

1 Create Student
id + name + getters

2 Create Course
code + title

3 Create CourseSection
Course + Student[] + count

4 Enroll
store references in next free slots

5 Print
iterate only from 0 to count-1

6 Trace
draw array + objects + shared Course

Prediction before execution: How many objects exist? Which references point to the same object?

Quick prediction activity

Do not run the code until everyone has drawn the reference picture

```
Course c = new Course("CSC241", "OOP");

CourseSection[] sections = new CourseSection[2];
    sections[0] = new CourseSection(c, "A");
    sections[1] = new CourseSection(c, "B");

        CourseSection x = sections[0];
               x = sections[1];
```

1. How many Course objects?

2. How many CourseSection objects?

3. How many array objects?

4. After the last line, what does x refer to?

5. Did sections[0] change?

Composition vs. inheritance: two very different questions

DESIGN CHOICE

Inheritance asks: IS-A?

Is every subtype truly a specialized form of the supertype?

Example: Car is a Vehicle.

Inheritance expresses substitutability and shared type behavior.

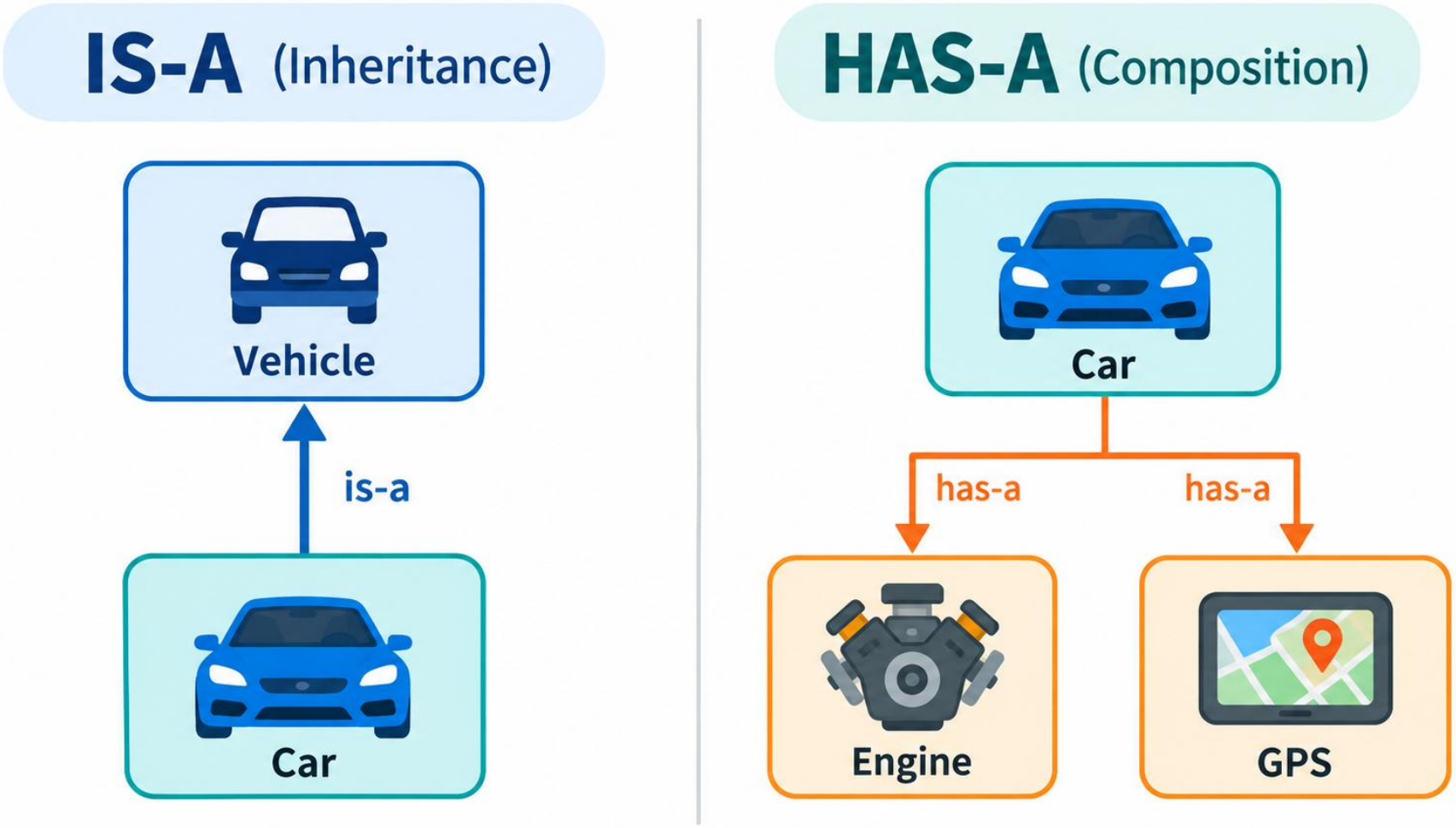
Composition asks: HAS-A / USES-A?

Does this object need another object to fulfill its responsibilities?

Example: Car has an Engine.

Composition expresses collaboration through object references.

Visual comparison — IS-A vs. HAS-A



A quick decision test

HEURISTIC

1. Can I naturally say “A is a B”?
2. Should an A be usable wherever a B is expected?
3. Or does A simply need / contain / collaborate with B?
4. Am I considering inheritance only to reuse code?

If the answer is “has/uses” rather than “is,” composition is usually the clearer model.

Bad inheritance example — CourseSection extends Course?

DESIGN SMELL

BAD IDEA

```
// Misleading model
class CourseSection extends Course {
    private String section;
}
```

Why it is wrong

A CourseSection is not a Course; it is an offering of a Course.

Substitution makes little domain sense.

A Course may have many sections.

Correct direction: CourseSection has a Course reference.

Bad inheritance example — Student extends Address?

DESIGN SMELL

Wrong reasoning

“Student needs address fields, so inherit from Address.”

This confuses code reuse with domain meaning.

A Student is not an Address.

BETTER

```
class Student {
    private Address address;   // HAS-A

    Student(Address address) {
        this.address = address;
    }
}
```

Composition gives flexible collaboration

DESIGN BENEFIT

Swap collaborators

A class can receive a different component object through its constructor.

The containing class does not need to change its inheritance hierarchy.

Limit assumptions

The class depends only on what it actually uses from the collaborator.

This often keeps relationships closer to domain meaning.

Easier evolution

New components can be introduced without claiming a new IS-A relationship.

Formal interface-based flexibility comes later in the course.

Refactor a data-heavy design into object composition

REFACTORING

BEFORE

```
// BEFORE: duplicate primitive data
class Enrollment {
    String studentId;
    String studentName;
    String courseCode;
    String courseTitle;
}
```

AFTER

```
// AFTER: collaborate with domain objects
class Enrollment {
    private Student student;
    private CourseSection section;

    Enrollment(Student student, CourseSection
section) {
        this.student = student;
        this.section = section;
    }
}
```

Object composition can replace duplicated primitive state with meaningful object references.

Live coding checkpoint 2 — Enrollment has Student + CourseSection

LIVE CODING

Build in class

Create Enrollment.java with two final reference fields.

Add a constructor that requires Student and CourseSection.

Add a method display() that asks both collaborators for information.

Compile after each small step.

START HERE

```
class Enrollment {
    private final Student student;
    private final CourseSection section;

    Enrollment(Student student, CourseSection section) {
        this.student = student;
        this.section = section;
    }
}
```

Compile and run the collaborating-object example

TERMINAL

POWERSHELL / TERMINAL

```
cd 00_Course_Master\01_Lectures\Week05\L09\LiveCoding

javac Student.java Course.java CourseSection.java Enr
java Main
```

Predict before execution

How many objects are created in Main?

Which object holds a reference to which?

Which references point to the same Course object?

What compiler error appears if CourseSection.java is omitted from compilation?

Trace the object graph — not just the output

PREDICT

MAIN.JAVA

```
Student s = new Student("BCS-001", "Abdul ur Rafe");
Course c = new Course("CSC241", "OOP");
CourseSection sec = new CourseSection(c, "A", 40);
Enrollment e = new Enrollment(s, sec);

Course again = sec.getCourse();
```

Questions

How many Course objects exist?

Does again == c evaluate to true?

Does Enrollment contain a copy of Student?

Which variables are references, and which objects live independently of those variable names?

Object composition and encapsulation work together

API DESIGN

Keep fields private

Reference fields are still internal state.

Do not expose them merely because they are objects.

Expose meaningful behavior

Prefer sec.getCourseCode() or sec.getLabel() if clients only need information.

Return the Course object only when collaboration is appropriate.

Protect invariants

Reject invalid null collaborators when the relationship is mandatory.

Avoid duplicate state that can disagree.

Common mistakes with object composition

MISCONCEPTIONS

Duplicate object data

Copy collaborator fields instead of keeping a reference.

Public reference fields

Clients can replace relationships without validation.

Confuse reference copy with object copy

Assignment copies the reference value, not the object.

Use inheritance for convenience

Code reuse alone is not an IS-A relationship.

Ignore null invariants

Required collaborators should not silently become null.

Return internals automatically

Decide API exposure deliberately.

In-class design activity — Library borrowing

TEAM ACTIVITY

A Loan connects one Member with one BookCopy. A Librarian creates the Loan after checking availability.

Task A — HAS-A

Which reference fields should Loan contain?

Which object should own dueDate?

Task B — method flow

Design borrow(Member m, BookCopy copy).

Which object(s) are passed to the method?

Task C — inheritance test

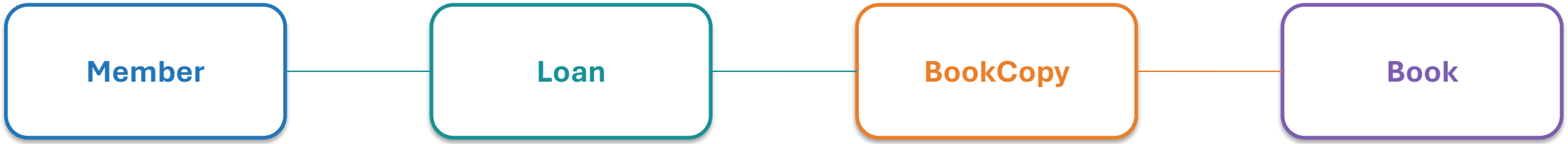
Should Loan extend Member? Why not?

Should BookCopy extend Book? Discuss carefully.

Deliverable: a 4-class sketch + one constructor + one method signature.

Activity debrief — one reasonable design

MODEL ANSWER



Loan

has Member, BookCopy, dueDate

Borrowing service

accepts Member + BookCopy arguments

Inheritance check

Loan is not Member; HAS-A is the right relationship.

Lecture recap & exit ticket

RECAP

Remember

Object arrays store references to objects; HAS-A relationships use object reference fields.

Objects can be passed to and returned from methods; Java copies reference values.

Mutation through a shared reference is visible; parameter reassignment is local.

Composition and inheritance express different domain meanings.

Prefer composition when the relationship is “has/uses,” not “is.”

Exit ticket

Why is CourseSection → Course a HAS-A relationship?

What is stored in each element of an object array, and what is copied when an object is passed?

Give one case where returning an internal object may weaken encapsulation.

Why is CourseSection extends Course a poor model?

Draw the object graph for Student → Enrollment → CourseSection → Course.

Next: Lecture 10 — multi-level composition, ownership/lifetime, cohesion/coupling, and stronger relationship semantics.